

# CSC1031

## OOP

Java Collections Framework Deep Dive

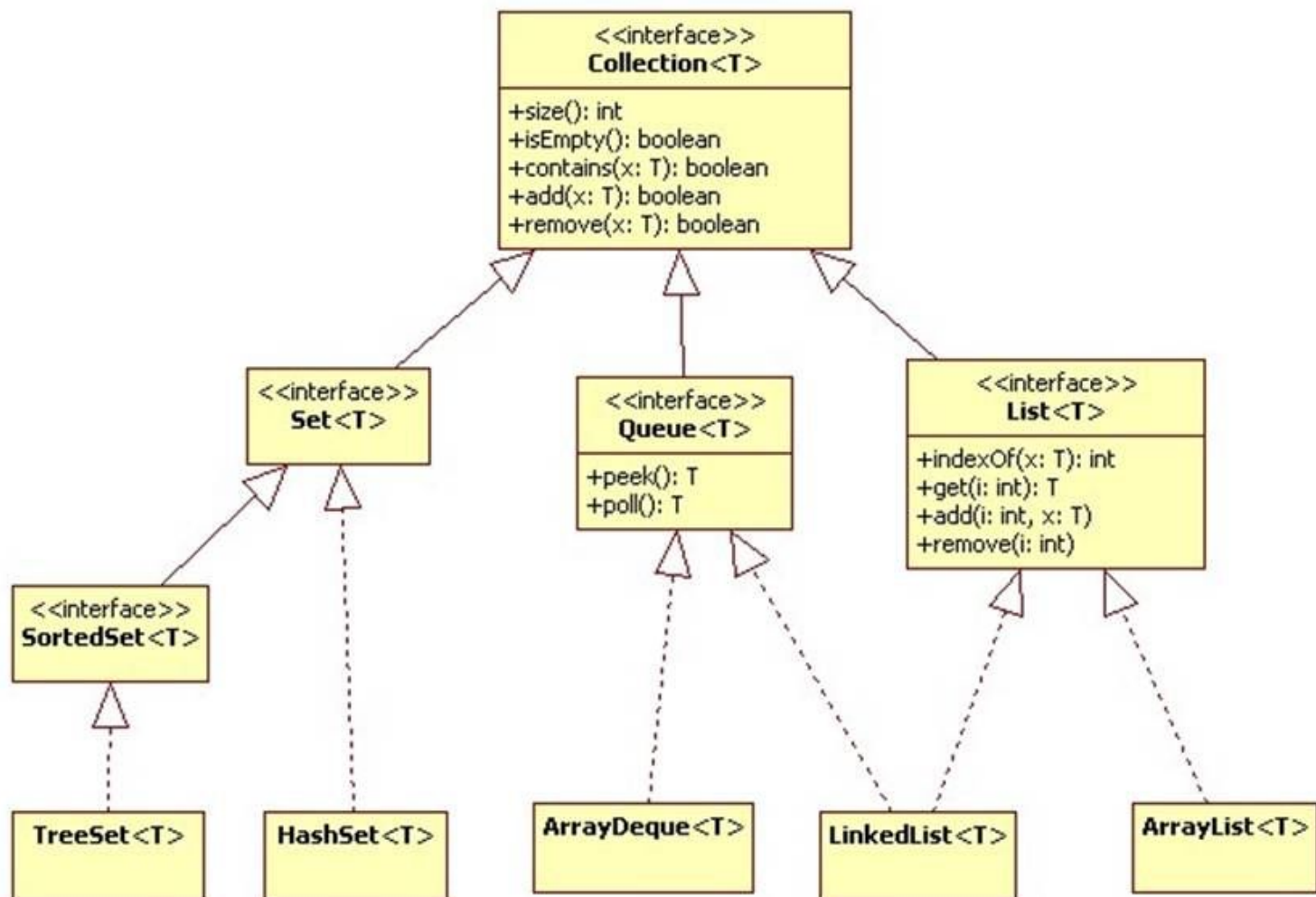


# Introduction to Java Collections Framework (JCF)

- The **Java Collections Framework (JCF)** provides reusable data structures.
- Designed to handle and manipulate groups of objects efficiently.
- Key interfaces:
  - `List`
  - `Set`
  - `Map`
  - `Queue`

## Why Use Collections Instead of Arrays?

- **Arrays:** Fixed size, not flexible.
- **Collections:** Dynamic, resizable, provide built-in operations.



# The **List** Interface

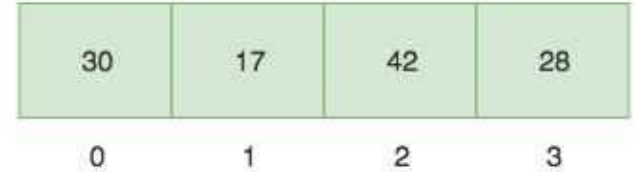
- **Ordered collection** allowing duplicate elements.
- Implementations:
  - `ArrayList`
  - `LinkedList`
  - `Vector`
  - `Stack`

# ArrayList vs. LinkedList

## ArrayList:

- Fast random access ( $O(1)$ )
- Efficient for read-heavy operations.
- Slow insertions & deletions in the middle ( $O(n)$ ).

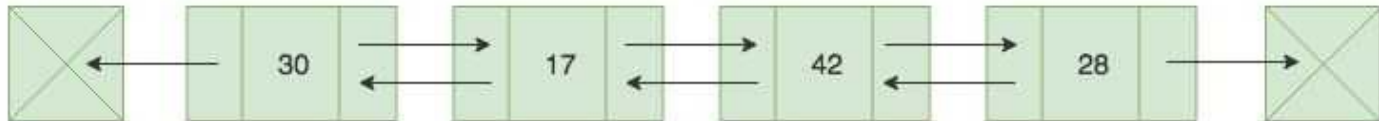
Java ArrayList  
Representation



## LinkedList:

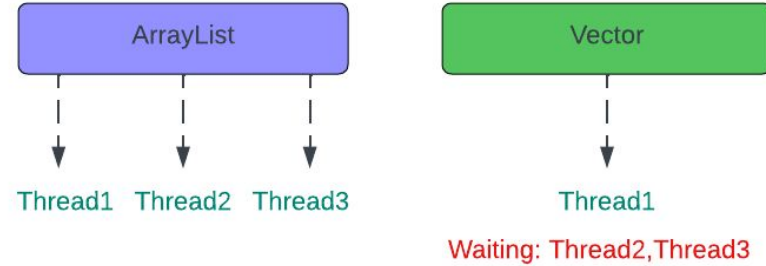
- Fast insertions & deletions ( $O(1)$  for head/tail).
- Slow random access ( $O(n)$ ).

Java LinkedList  
Representation

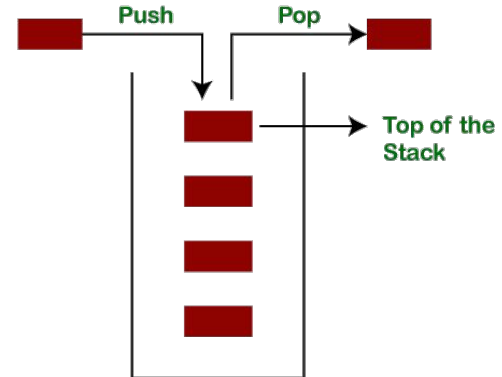


# Vector and Stack

- `Vector<E>` is a **resizable array** in Java that implements the `List` interface.
- It is **synchronized**, making it **thread-safe** but slower than `ArrayList`.
- Used when **multiple threads access** the same collection.



- `Stack<E>` is a **LIFO (Last In, First Out)** data structure.
- Extends `Vector<E>` but adds additional methods for stack operations.
- Useful for **backtracking, undo mechanisms, and expression evaluation**.
- `Stack<E>` extends `Vector<E>`, meaning it inherits synchronization.
- Implements push-pop operations for stack-based behavior.



## The Set Interface

- **Unique elements only** (no duplicates).
- Implementations:
  - HashSet
  - TreeSet
  - LinkedHashSet



## HashSet vs. TreeSet vs. LinkedHashSet

### HashSet:

- Fastest ( $O(1)$  average time for insert, delete, search).
- No ordering of elements.

### TreeSet:

- Maintains **sorted order**.
- Slower ( $O(\log n)$  for insert, delete, search).

### LinkedHashSet:

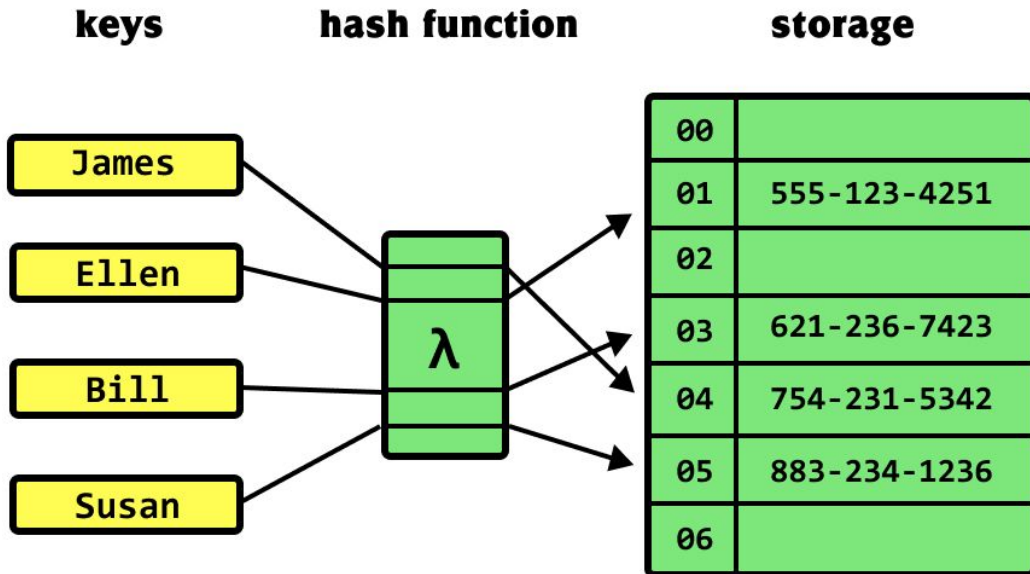
- Insertion order
- Slightly slower than HashSet ( $O(1)$ )
- Use **LinkedHashSet** when you need **fast access** while **preserving insertion order**.

# How Hashing Works in Java

Hashing is a process of **converting data into a fixed-size integer** (hash code).

A **hash function** takes an input (e.g., a string) and maps it to a number.

Java uses **hash codes** to store and retrieve data efficiently in **HashSet**, **HashMap**, and **HashTable**.

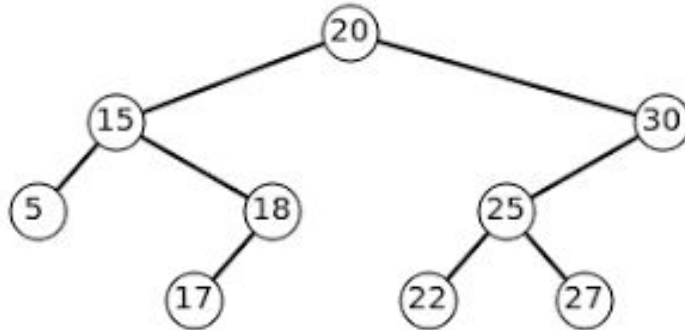


# How TreeSet Works in Java

`TreeSet<E>` is a **sorted set** implementation in Java.

It is based on a **Red-Black Tree**, a **self-balancing binary search tree**.

Maintains elements in **ascending order** by default.



## The Map Interface

- **Key-value pairs.**
- Implementations:
  - `HashMap`
  - `TreeMap`
  - `LinkedHashMap`

| Key         | Value |
|-------------|-------|
| "apple"     | 4     |
| "cherry"    | 2     |
| "pecan"     | 5     |
| "blueberry" | 1     |

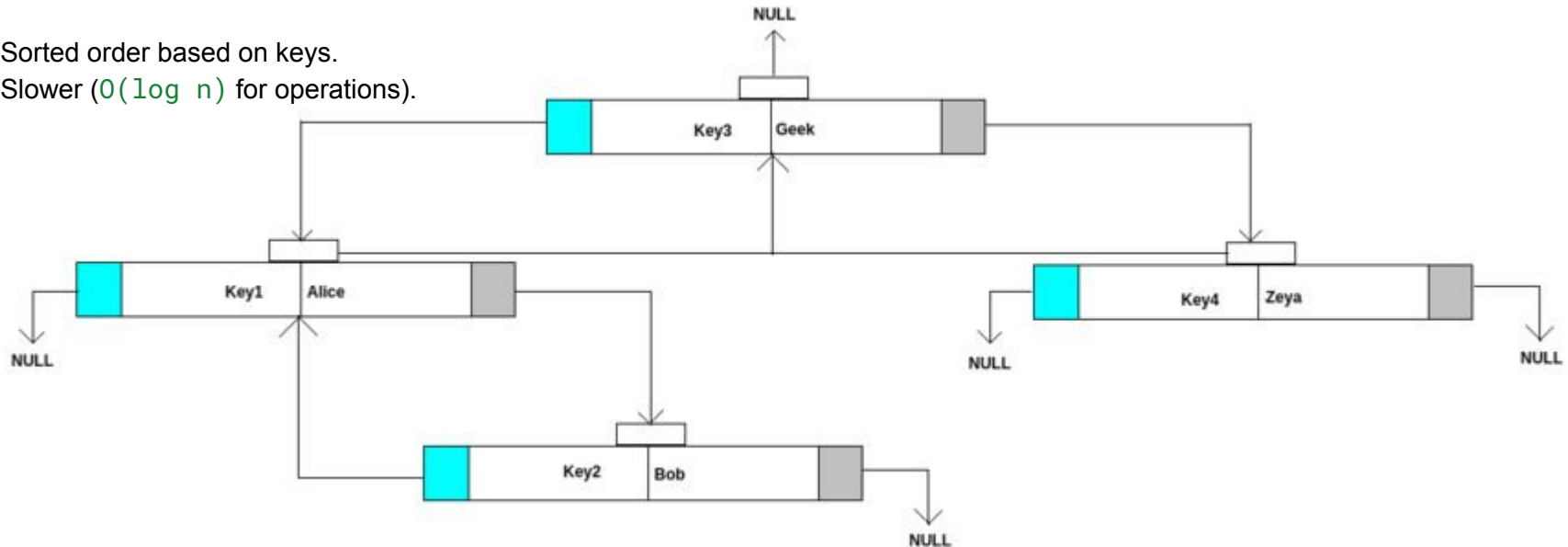
## HashMap vs. TreeMap

### HashMap:

- Fastest key lookup ( $O(1)$  average).
- No ordering of keys.
- **Not thread-safe**

### TreeMap:

- Sorted order based on keys.
- Slower ( $O(\log n)$  for operations).



## The Queue Interface

- **FIFO (First-In-First-Out)** structure.
- Implementations:
  - `LinkedList` (as a queue)
  - `PriorityQueue`
  - `ArrayDeque`



## Priority Queue Example

`PriorityQueue<E>` is a **heap-based** queue where elements are **ordered by priority** instead of insertion order.

Provides  **$O(\log n)$**  time complexity for insertion and deletion.

**By Default:** Implements a **Min-Heap**, where the **smallest element is at the head**.

**Custom Order:** You can use a **Comparator** to define custom priority rules.

```
Comparator<String> stringLengthComparator = new
Comparator<String>() {
    @Override
    public int compare(String s1, String s2) {
        return s1.length() - s2.length();
    }
};
```

```
// Create a Priority Queue with a custom Comparator
PriorityQueue<String> namePriorityQueue = new
PriorityQueue<>(stringLengthComparator);
```

```
// Add items to a Priority Queue (ENQUEUE)
namePriorityQueue.add("Lisa");
namePriorityQueue.add("Robert");
namePriorityQueue.add("John");
namePriorityQueue.add("Chris");
namePriorityQueue.add("Angelina");
namePriorityQueue.add("Joe");
```

# Output  
Joe  
John  
Lisa  
Chris  
Robert  
Angelina

## Performance Comparison

| Collection | Lookup      | Insert      | Delete      |
|------------|-------------|-------------|-------------|
| ArrayList  | $O(1)$      | $O(n)$      | $O(n)$      |
| LinkedList | $O(n)$      | $O(1)$      | $O(1)$      |
| HashSet    | $O(1)$      | $O(1)$      | $O(1)$      |
| TreeSet    | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |
| HashMap    | $O(1)$      | $O(1)$      | $O(1)$      |
| TreeMap    | $O(\log n)$ | $O(\log n)$ | $O(\log n)$ |

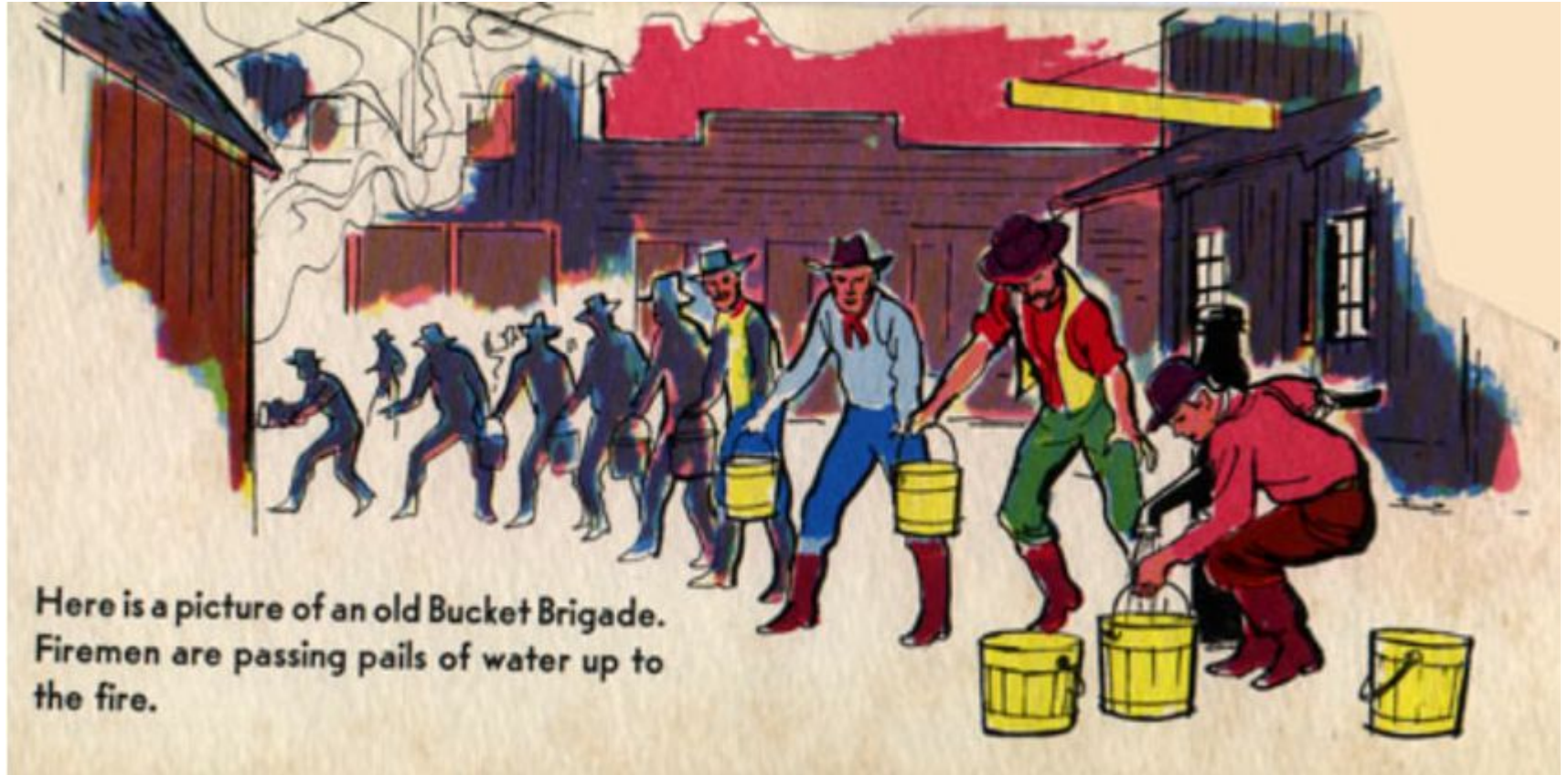


## Choosing the Right Collection

- Use **ArrayList** when fast random access is needed.
- Use **LinkedList** when frequent insertions/deletions are required.
- Use **HashSet** for unique elements when order doesn't matter.
- Use **TreeSet** when elements need to be sorted.
- Use **HashMap** when key-based lookup is needed.
- Use **TreeMap** when sorted keys are required.

Design pattern

## Chain of Responsibility



# What is Chain of Responsibility?

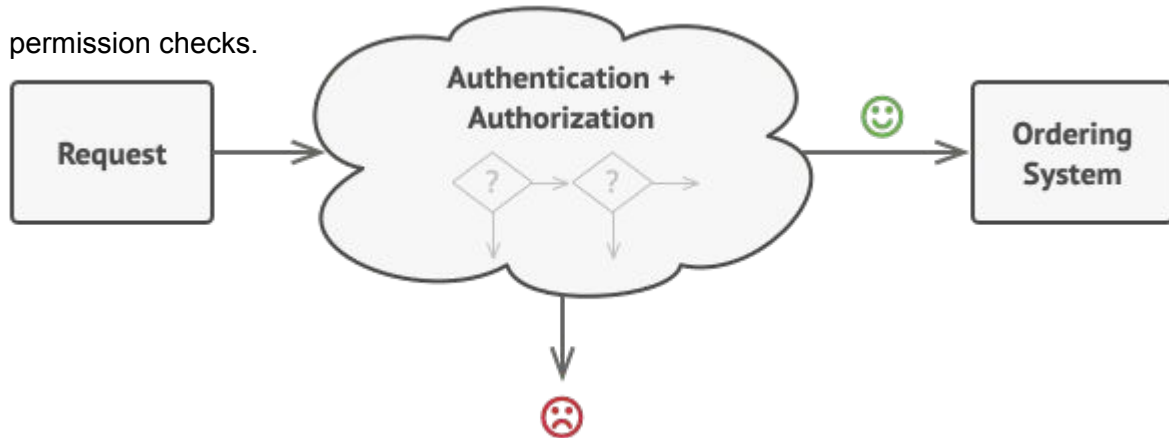
- A **behavioral design pattern** that allows passing requests through a **chain of handlers**.
- Each handler **decides** whether to process the request or forward it to the next handler.

## Problem in an Online Ordering System

Example:

**Access Control:** Only authenticated users can create orders.

- **First step:** Authenticate user credentials.
- **If authentication fails:** Stop processing.
- **If authentication succeeds:** Continue with permission checks.
- **Payment checks** and so on



# What is Chain of Responsibility?

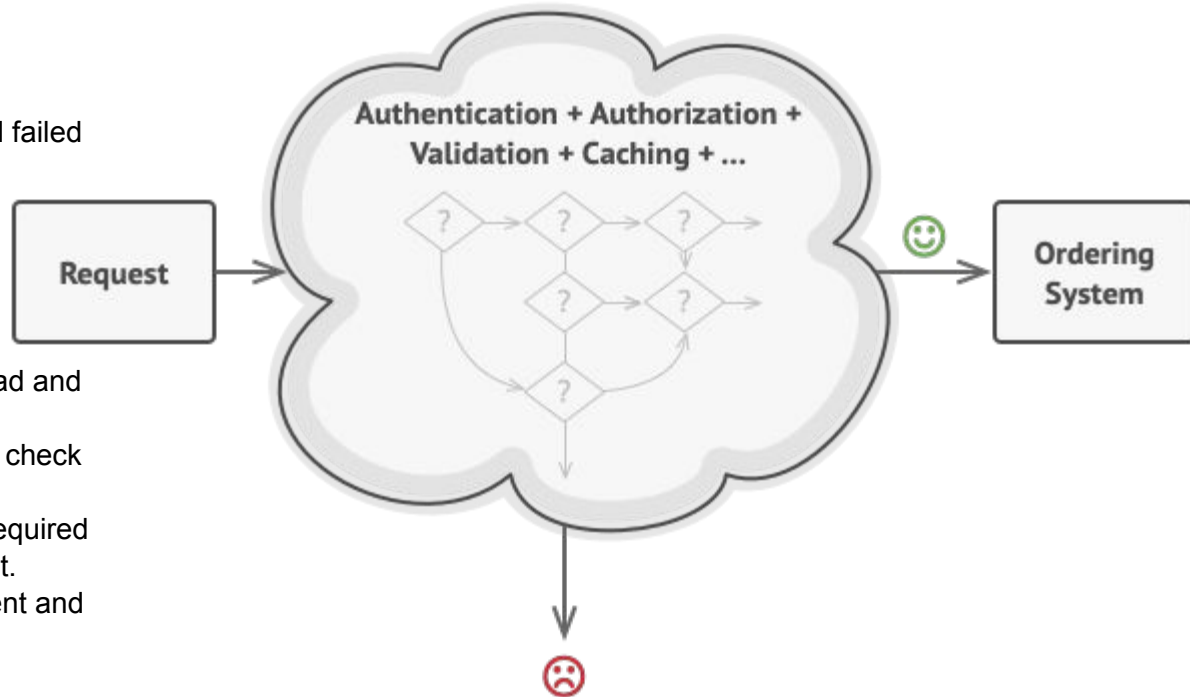
## Growing Complexity in the Ordering System

### Added New Checks Over Time:

- **Data Validation:** Sanitizes input before processing.
- **Brute Force Protection:** Blocks repeated failed login attempts.
- **Caching Mechanism:** Returns cached responses for repeated requests.

### Challenges Faced:

- **Messy and bloated code** – Difficult to read and modify.
- **Tightly coupled checks** – Changing one check impacted others.
- **Code duplication** – Some components required only specific checks, making reuse difficult.
- **Hard to maintain** – Increased development and debugging costs.



# Solution

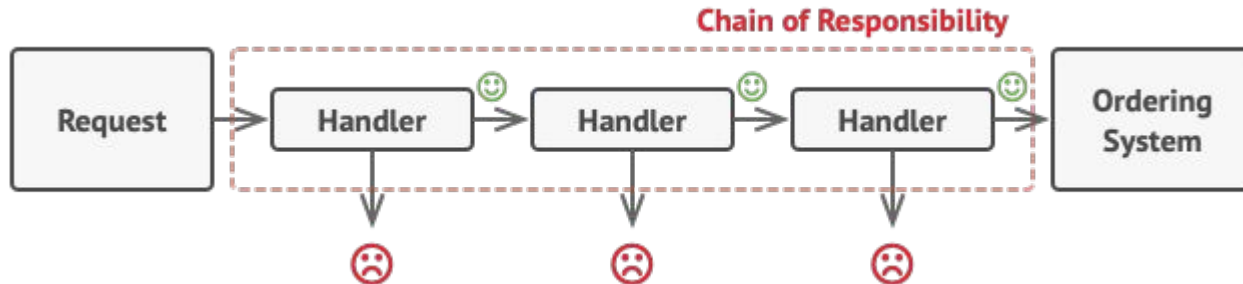
**Handlers** are transformed into **stand-alone objects**, each responsible for a single check.

The **request is passed** to each handler in sequence, allowing independent processing.

**Handlers are linked together in a chain.**

Each handler:

- **Processes the request** (e.g., authentication, validation, caching).
- **Decides whether to pass it further** or stop execution.
- The request **travels through the chain** until all relevant checks are completed.



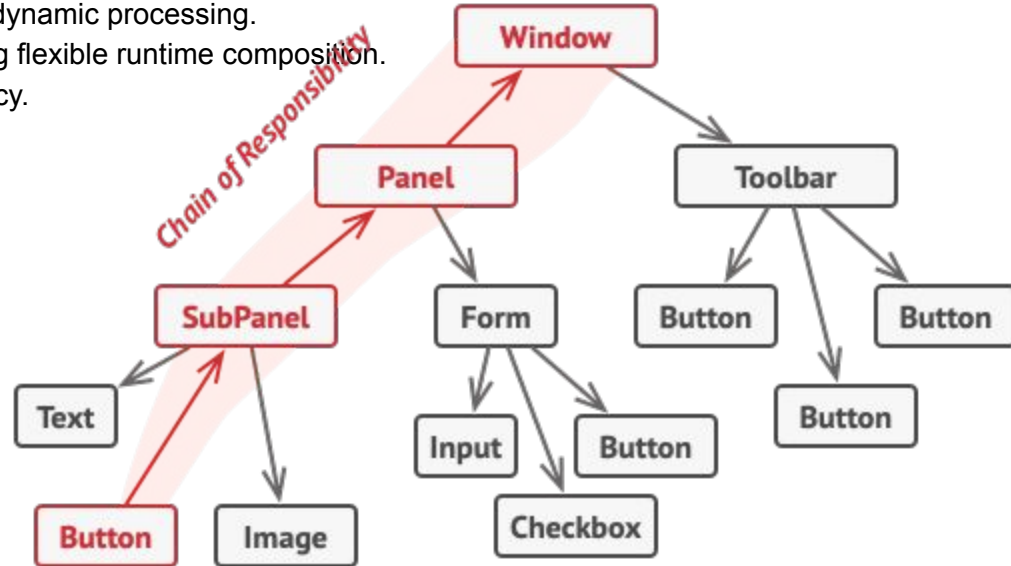
# Alternative Approach in Chain of Responsibility

Key Concept: **First Handler Wins**

- Instead of all handlers processing the request, **only the first capable handler acts**, then stops the chain.
- Common in **GUI event handling**, where events propagate **until handled**.

## Flexible Chain Formation

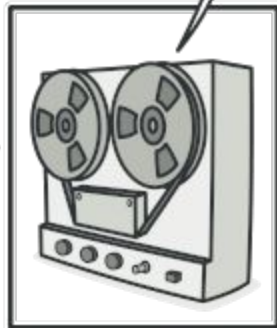
- **A chain can be formed from an object tree**, allowing dynamic processing.
- **All handlers implement a common interface**, enabling flexible runtime composition.
- **Prevents unnecessary processing**, improving efficiency.



HELLO,  
I NEED SOME  
TECH SUPPORT.



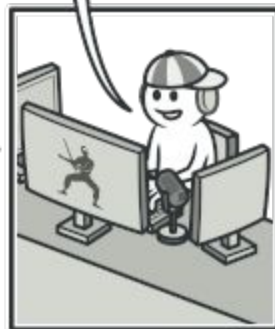
PRESS 1, IF ...



HAVE YOU TRIED  
TURNING IT OFF  
AND ON AGAIN?



OK, HERE'S  
WHERE YOU  
CAN DOWNLOAD  
SOME DRIVERS.

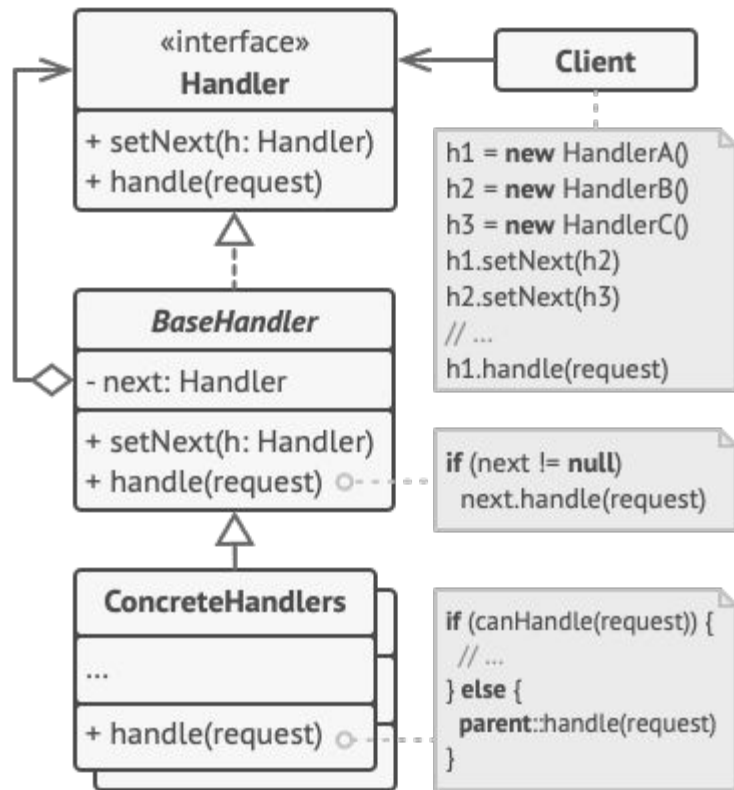


# Structure

The **Handler** declares the **interface**, common for all concrete handlers. It usually **contains just a single method** for handling requests, but sometimes it may also have another method for setting the next handler on the chain.

The **Base Handler** is an **optional** class where you can put the boilerplate **code that's common to all handler** classes.

Usually, this class defines a field for storing a **reference to the next handler**. The clients can build a chain by passing a handler to the constructor or setter of the previous handler. The class may also implement the default handling behavior: it can pass execution to the next handler after checking for its existence.

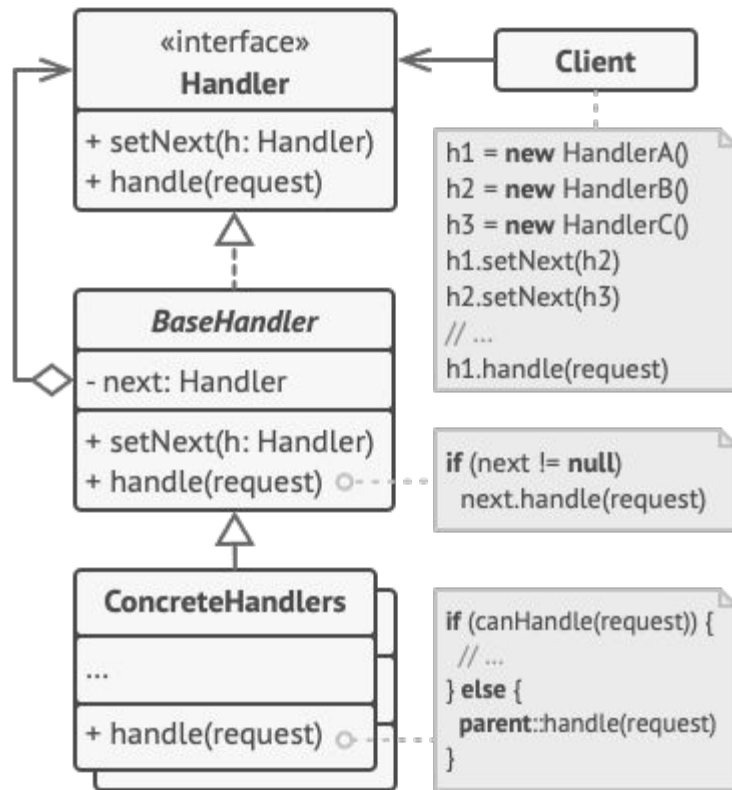




# Structure

The **Client** may **compose chains** just once or compose them dynamically, depending on the application's logic. Note that a **request can be sent to any handler in the chain**—it doesn't have to be the first one.

**Concrete Handlers** contain the **actual code** for processing requests. Upon receiving a request, each handler must decide whether to process it and, additionally, whether to pass it along the chain.



# When to Use Chain of Responsibility

## Processing Different Types of Requests

- Useful when the **request types and order are unknown beforehand**.
- Allows dynamic decision-making on **which handler processes the request**.

## Executing Multiple Handlers in Order

- Ensures requests pass **through a predefined sequence** of handlers.
- Handlers can be **linked in any order**, giving full control over execution flow.

## Dynamic Modification of Handlers at Runtime

- Handlers can be **inserted, removed, or reordered** dynamically.
- Using **setters for handler references** enables flexible chain composition.

# Pros and Cons

- + You can control the order of request handling.
- + Single Responsibility Principle. You can decouple classes that invoke operations from classes that perform operations.
- + Open/Closed Principle. You can introduce new handlers into the app without breaking the existing client code.
- Some requests may end up unhandled.